Thanks to creative tricks such as this one, Camping feels less like a Ruby web framework and more like a domain-specific language for web development. In general, this is a good thing, as I argue in Appendix 2, *Domain-Specific Languages*, on page 227.

Nil Guards

Most Ruby beginners looking through someone else's code are perplexed by this exotic idiom:

common_idioms/nil_guards.rb

```
a ||= []
```

In this example, the value to the right happens to be an empty array, but it could be any assignable value. The ||= is actually a syntax shortcut for the following:

```
a || (a = [])
```

To understand this code, you need to understand the details of the "or" operator (||). Superficially, the || operator simply returns true if either of the two operands is true—but there is some subtlety to this. Here's the way that || actually works.

Remember that in a Boolean operation, any value is considered true with the exception of nil and false. If the first operand is true, then || simply returns the first operand, and the second operand is never evaluated. If the first operand is *not* true, then || evaluates and returns the second operand instead. This means the result will be true unless both operands are false, which is consistent with the intuitive notion of an or operator.

Now you can see that the previous code has the same effect as this:

```
if defined?(a) && a
  a
else
  a = []
end
```

You can translate this code as this: "If a is nil, or false, or hasn't even been defined yet, then make it an empty array and give me its value; if it's anything else, just give me its value." In such cases, experienced Ruby coders generally consider the ||= operator more elegant and readable than an if. You're not limited to arrays, so you can use the same idiom to initialize just about anything. This idiom is sometimes called a *Nil Guard*, because it's used to make sure that a variable is not nil.

Spell: Nil Guard